

Resumen Teorico – Practico 7

Testing basado en propiedades y generacion aleatoria de tests

Testing de Software 2024

Índice

Introduccion	3
1. Por que ir mas alla del unit testing	3
1.1. El espectro garantia vs. esfuerzo	3
1.2. Problemas de la creacion manual de tests	4
2. Property-Based Testing	4
2.1. Definicion	4
2.2. Estructura de una propiedad	4
2.3. Ejemplo: <code>BoundedQueue.enqueue</code>	5
2.4. Mas ejemplos clasicos de propiedades	5
2.5. Tipos de propiedades utiles	6
2.6. Como hace el framework para muestrear	6
2.7. Shrinking: minimizar contraejemplos	7
3. Generadores	7
3.1. Por que importan tanto	7
3.2. Primitivos: enteros, floats, strings	7
3.3. Combinadores	7
3.4. Generadores dependientes y validez	8
3.5. <code>filter</code> vs. <code>flatMap</code>	8
4. Generacion automatica de tests	8
4.1. Idea general	8

4.2. Generacion aleatoria guiada por feedback	9
4.3. Oraculo	9
4.4. Por que feedback	10
4.5. Limitaciones	10
5. Relacion entre PBT, fuzzing y generacion automatica	10
6. Patrones para disenar propiedades	11
6.1. El patron “oracle por construccion”	11
6.2. El patron “operaciones inversas”	11
6.3. El patron “invariante de representacion”	11
6.4. El patron “ley algebraica”	11
7. Comparacion con el resto del curso	11
8. Glosario	12
Conclusion	12

Introduccion

Este resumen sintetiza el material teorico asociado al TP7:

- *Notas 12: Testing basado en propiedades* (V. Bengolea).
- *Notas 13: Generacion automatica de tests — generacion aleatoria guiada por feedback* (V. Bengolea).

Idea general. Hasta aqui los tests se escribian uno a uno: cada `@Test` fijaba un input concreto y verificaba una salida concreta. Esa estrategia es valiosa pero *costosa, tediosa e incompleta*. Las dos tecnicas que se cubren en el TP7 amplian el espectro:

- **Property-Based Testing (PBT):** en lugar de un input concreto, se enuncia una *propiedad* que debe valer para todas las entradas dentro de una precondition. El framework muestrea valores y reporta cualquier contraejemplo.
- **Generacion aleatoria automatica:** la herramienta no solo muestrea inputs, sino que construye *secuencias de operaciones* aleatorias sobre las APIs del programa, alimentando *feedback* de las ejecuciones anteriores para guiar la busqueda.

1. Por que ir mas alla del unit testing

1.1. El espectro garantia vs. esfuerzo

Las tecnicas de verificacion se pueden ordenar en un eje “cuanta garantia dan” vs. “cuanto esfuerzo cuestan”:

1. **Metodos tradicionales** (unit tests escritos a mano): bajo esfuerzo por test, garantia muy local.
2. **Testing automatico** (PBT, fuzzing, generacion aleatoria): mas garantia con esfuerzo medio. El programador solo escribe propiedades o configura generadores.
3. **Metodos formales automaticos** (model checking, bounded verification): mayor garantia, costo de configuracion alto.
4. **Verificacion deductiva** (Hoare, Coq, Isabelle): garantia maxima pero esfuerzo enorme.

Idea clave

PBT y generacion automatica viven en el “escalon util” del espectro: dan mas garantia que los unit tests sin pedir el esfuerzo de un metodo formal. Bien usados, pueden encontrar bugs sutiles que ningun unit test individual cubriria.

1.2. Problemas de la creacion manual de tests

Escribir tests a mano tiene tres problemas estructurales:

- **Es costosa.** Cada caso requiere pensar input, oraculo y assertion.
- **Es tediosa.** Muchos casos son variaciones triviales.
- **Es incompleta.** El humano tiende a probar los casos que ya tiene en mente; los bordes y combinaciones inusuales suelen quedar fuera.

La automatizacion ataca los tres: testing automatico genera muchos casos rapidamente, y explora regiones del input que un humano dificilmente cubriria.

2. Property-Based Testing

2.1. Definicion

Definicion

Un **test basado en propiedades** (Property-Based Test) captura un aspecto del comportamiento del sistema bajo test para un *numero potencialmente infinito de entradas*. En lugar de un par fijo (input, output esperado), se enuncia una propiedad logica que debe valer para *todas* las entradas dentro de una precondition.

Contraste con unit testing.

- **Unit test:** comportamiento para una entrada particular.
- **Property test:** comportamiento para una familia entera de entradas.

2.2. Estructura de una propiedad

Una propiedad tipicamente sigue el esquema:

```
public void myProperty(MyEntry e) {  
    AssumePrecondition(e);    // 1. Precondicion: descartar  
        entradas invalidas  
    Arrange(e);                // 2. Preparar el escenario (  
        opcional)  
    Act(e);                    // 3. Ejecutar la operacion bajo  
        test  
    AssertExpectedProperty(e); // 4. Postcondicion: verificar la  
        propiedad  
}
```

Las cuatro partes:

- **Precondicion (assume).** Describe las entradas validas. Si el generador produce algo que no la cumple, el caso se descarta (no se considera “fail”).
- **Arrange.** Setup necesario antes de la operacion (e.g. poblar una lista).
- **Act.** La operacion bajo prueba.
- **Postcondicion (assert).** Lo que el codigo debe garantizar, expresado en funcion de los parametros que cumplen la precondicion.

2.3. Ejemplo: BoundedQueue.enqueue

Propiedad. Para toda cola no llena, insertar un elemento incrementa el tamaño en 1.

Unit test:

```
@Test public void enqueueSizeTest_1() {
    BoundedQueue q = new BoundedQueue();
    q.enqueue(4); q.enqueue(3); q.enqueue(5);
    assertThat(q.size(), is(3));
}
```

Este test verifica la propiedad para un escenario puntual. Necesitaríamos muchos tests parecidos para distintos tamaños iniciales y elementos.

Property test:

```
public void enqueueSizeProperty(BoundedQueue queue, Integer i) {
    assumeTrue("cola no llena", !queue.isFull());
    int oldSize = queue.size();
    queue.enqueue(i);
    assertThat(queue.size(), is(oldSize + 1));
}
```

Una sola propiedad reemplaza a una familia entera de unit tests, y el framework muestrea muchos valores de `queue` y de `i`.

2.4. Mas ejemplos clasicos de propiedades

- **Sets:** “si se agrega un elemento que ya pertenece, el set no cambia”.

$$\forall s, x. x \in s \Rightarrow s \cup \{x\} = s.$$

- **Strings:** “ a es substring de `concat( $a, b$ )` para todo a, b ”.
- **Math.max:** para todo a, b enteros,
 - `Math.max(a, b) == Math.max(b, a)` (conmutatividad),
 - `Math.max(a, b) >= a` y `>= b`,
 - `p + Math.max(a, b) == Math.max(p+a, p+b)` (compatibilidad con suma).

2.5. Tipos de propiedades utiles

- **Invariantes.** Propiedades que se preservan al aplicar operaciones (e.g. `repOK()` antes y despues).
- **Contratos del lenguaje.** Como el contrato `equals/hashCode` de `Object`: si `a.equals(b)` entonces `a.hashCode() == b.hashCode()`.
- **Propiedades algebraicas.** Conmutatividad, asociatividad, distributividad, idempotencia.
- **Round-trip.** `deserialize(serialize(x)) == x`, `parse(print(t)) == t`, `decompress(compress(d)) == d`.
- **Oraculo de referencia.** Si existe una implementacion lenta pero confiable, la propiedad puede ser “mi implementacion rapida da el mismo resultado que la lenta”.
- **Metamorficas.** Relacion entre dos llamadas del mismo metodo: `sort(reverse(l)) == sort(l)`.

2.6. Como hace el framework para muestrear

Un framework PBT (e.g. `jqwik` en Java, `QuickCheck` en Haskell, `Hypothesis` en Python) ejecuta cada propiedad cientos de veces. En cada iteracion:

1. Llama a los generadores (`@ForAll/@Provide`) para producir valores para los parametros.
2. Aplica la precondition (`assumeTrue`); si falla, descarta el caso y prueba otro.
3. Ejecuta la propiedad. Si lanza una excepcion o falla una `assertion`, registra el contraejemplo.

2.7. Shrinking: minimizar contraejemplos

Definicion

Shrinking. Cuando una propiedad falla, el framework intenta *simplificar* el input que provoco la falla para obtener el contraejemplo *minimo*. Por ejemplo, si una lista de 100 elementos rompe la propiedad, el shrinking busca la sub-lista mas chica que aun la rompe.

Idea clave

Un contraejemplo minimo es muchisimo mas facil de depurar que uno gigante con datos al azar. Por eso shrinking es lo que distingue a un framework PBT serio de un simple “ejecuta esto mil veces”.

3. Generadores

3.1. Por que importan tanto

Las propiedades son tan buenas como los generadores. Un generador que solo produce listas vacias hace que la propiedad “ordenar preserva la longitud” sea trivialmente verdadera. Hay que pensar:

- Que entradas son *validas*.
- Que entradas son *representativas*.
- Que entradas son *borde* (valores extremos, vacios, repetidos).

3.2. Primitivos: enteros, floats, strings

Casi todos los frameworks proveen primitivos con rangos configurables:

```
Arbitraries.integers().between(0, 100);  
Arbitraries.floats().between(-1e3f, 1e3f);  
Arbitraries.strings().withCharRange('a', 'z').ofMaxLength(20);
```

3.3. Combinadores

A partir de primitivos se arman generadores compuestos. Los dos mas importantes son `map` y `flatMap`:

- **map:** aplica una funcion al valor generado.

```
Arbitraries.integers().between(0, 100).map(n -> "user" + n);
```

- **flatMap**: permite que un parametro *dependa* de otro.

```
// generador de listas y un indice valido sobre cada lista
Arbitraries.integers().between(1, 100).flatMap(size ->
  Arbitraries.integers().list().ofSize(size).flatMap(list ->
    Arbitraries.integers().between(0, size - 1).map(idx ->
      new Scenario(list, idx)))));
```

3.4. Generadores dependientes y validez

Muchos dominios requieren *validez compuesta*: un dia depende del mes y del ano, un indice depende del tamaño de la lista, una transicion de FSM depende del estado actual. **flatMap** es la herramienta para construir esos generadores sin tener que filtrar muchas entradas invalidas.

Ejemplo

Generar fechas validas a partir de 1900:

```
Arbitraries.integers().between(1900, 2400).flatMap(year ->
  Arbitraries.integers().between(1, 12).flatMap(month ->
    Arbitraries.integers().between(1, daysInMonth(month, year)
    ))
  .map(day -> new Date(day, month, year))));
```

Sin **flatMap** se generarian fechas como 31/04/2024, que el constructor rechazaria.

3.5. filter vs. flatMap

filter descarta valores que no cumplen una condicion. Es *lo ultimo* a usar: si la condicion es estricta (por ejemplo, “primos”), el framework descartara miles de muestras hasta encontrar una valida.

Idea clave

Regla practica: si el filtro descarta mas del ~30 % de las muestras, conviene rediseñar el generador con **flatMap** o construir directamente valores validos.

4. Generacion automatica de tests

4.1. Idea general

La generacion aleatoria automatica de tests va un paso mas alla del PBT: en vez de pedirle al programador que escriba propiedades y generadores, la herramienta:

1. Recibe el código del programa (sus clases, métodos, constructores).
2. Genera *secuencias de llamadas* aleatorias a esas APIs.
3. Ejecuta cada secuencia y observa que pasa (excepciones, valores devueltos, contratos violados).
4. Usa esa información como *feedback* para decidir como continuar.

4.2. Generación aleatoria guiada por feedback

Definición

Generación guiada por feedback (e.g. *Randoop*). Se genera la secuencia de llamadas paso a paso, ejecutando cada paso inmediatamente. Si la ejecución lanza una excepción no contractual (e.g. un `NullPointerException` no esperado), la secuencia se reporta como *test que muestra un bug*. Si la ejecución va bien, el resultado se guarda en un *pool* de valores que puede usarse para construir secuencias futuras.

Ciclo básico:

1. Comenzar con un pool de valores trivial (enteros, strings).
2. Elegir un método al azar de la API bajo prueba.
3. Elegir, del pool, valores que sirvan como argumentos compatibles en tipos.
4. Ejecutar la llamada:
 - Si la ejecución falla con un crash inesperado, reportar la secuencia como test sospechoso.
 - Si retorna un valor, agregarlo al pool.
 - Si lanza una excepción documentada, marcar como “no extendible” y descartar.
5. Repetir hasta agotar el presupuesto de tiempo.

4.3. Oráculo

Un detalle clave: *quien decide* si una ejecución es un bug? Hay varios oráculos posibles:

- **Crashes inesperados.** `NullPointerException`, `ArrayIndexOutOfBoundsException`, `AssertionError` en lugares donde la especificación no los permite.
- **Contratos del lenguaje.** `equals/hashCode/toString/compareTo` con sus reglas: reflexividad, simetría, transitividad, etc.

- **Aserciones internas** del código (`assert` de Java).
- **Diferencial.** Comparar contra una versión de referencia.

4.4. Por que feedback

La aleatoriedad pura genera muchas secuencias *redundantes* (mismas operaciones, mismos valores). La guía por feedback hace varias cosas:

- Evita repetir secuencias equivalentes (memoization de pools).
- Acumula objetos en estados *interesantes* que luego pueden combinarse.
- Detecta secuencias que terminan rápido (excepciones esperadas) y las descarta como “muertas”.

4.5. Limitaciones

- **Oráculo limitado.** Sin propiedades explícitas, la herramienta solo detecta bugs “visibles” (crashes, contratos de lenguaje). Las violaciones de lógica de dominio pasan inadvertidas.
- **Tests poco legibles.** Los tests generados son secuencias mecánicas; cuando fallan, son difíciles de entender.
- **Cobertura sesgada.** La aleatoriedad uniforme no es exploratoria: ciertas partes del código quedan profundamente cubiertas y otras no.

5. Relación entre PBT, fuzzing y generación automática

Técnica	Quien escribe que	Que entrega
Unit testing	Programador escribe input y oráculo	Tests concretos
PBT	Programador escribe propiedad y generador	Familia de tests
Fuzzing	Programador da semillas u oráculo mínimo	Inputs (rara vez tests legibles)
Generación automática	Programador da API (o nada)	Secuencias de llamadas

Idea clave

A medida que se baja en la tabla, el programador escribe menos pero *el oraculo se debilita*. PBT es un buen punto medio: la propiedad da un oraculo fuerte (no solo “no crashea”), y el generador da exploracion amplia.

6. Patrones para disenar propiedades

6.1. El patron “oracle por construccion”

Cuando es dificil enunciar el resultado correcto, a veces se puede construir el escenario *con* el resultado ya conocido:

Ejemplo

En vez de generar un grafo y verificar “el camino mas corto entre u y v ”, se construye un grafo donde ya se sabe el camino mas corto (por ejemplo, un grafo con un solo camino posible) y se verifica que el algoritmo lo encuentra.

6.2. El patron “operaciones inversas”

Si una operacion tiene una inversa (deserializar/serializar, encriptar/desencriptar, parse/-print, push/pop), la propiedad de round-trip es muy efectiva: aplica ambas y verifica que se vuelve al estado inicial.

6.3. El patron “invariante de representacion”

Pedir que `repOK()` se preserve por toda operacion publica. Captura una clase entera de bugs (corrupcion de estado interno) con una propiedad sola.

6.4. El patron “ley algebraica”

Si el dominio tiene leyes (asociatividad, conmutatividad, identidad), enunciar cada ley como una propiedad.

7. Comparacion con el resto del curso

- **Particion del input** (TP2) y PBT son complementarias: PBT muestrea *en* cada bloque, pero la particion ayuda a definir los rangos del generador.
- **Cobertura estructural** (TP3-TP4) responde a una pregunta distinta: “¿toque todo el codigo?”. PBT responde “¿mi codigo cumple la propiedad?”. Idealmente ambas se

combinan: usar cobertura para asegurarse de que los generadores ejercitan todas las ramas.

- **Cobertura logica** (TP5) y PBT se relacionan: las propiedades suelen escribirse como predicados, y los generadores pueden diseñarse para ejercitar CACC de esos predicados.
- **Mutation testing** (TP6) puede medir la calidad de una suite de propiedades: un buen conjunto de propiedades debería matar la mayoría de los mutantes igual que una buena suite de unit tests.
- **Fuzzing** (TP6) es generacion aleatoria *sin* propiedades sofisticadas: el oraculo es “no crash”. PBT y generacion guiada por feedback agregan oraculos mas precisos.

8. Glosario

- **Propiedad**: enunciado que debe valer para todas las entradas que cumplan una pre-condicion.
- **Generador** (*arbitrary, provider*): mecanismo que produce valores aleatorios para un parametro.
- **Shrinking**: proceso de minimizar un contraejemplo cuando una propiedad falla.
- **Precondicion**: condicion sobre las entradas para que la propiedad sea aplicable.
- **Postcondicion**: lo que el codigo debe garantizar.
- **Oraculo**: mecanismo que decide si la ejecucion es correcta.
- **Pool de valores**: conjunto de objetos disponibles para construir nuevas secuencias en generacion automatica.

Conclusion

PBT cambia la unidad de testing: en vez de pensar en *un* input, se piensa en una *familia* entera de inputs. Esa abstraccion fuerza al programador a hacer explicitas las propiedades que su codigo realmente debe satisfacer — y muchas veces, al hacerlo, se descubre que las propiedades no estaban tan claras. La generacion automatica de tests empuja aun mas la automatizacion: con una API y un oraculo minimo, la herramienta busca secuencias que crasheen. Ninguna de las dos reemplaza al unit testing, pero ambas atacan problemas que el unit testing solo no resuelve: cobertura sesgada, casos borde no-pensados y bugs combinatorios.